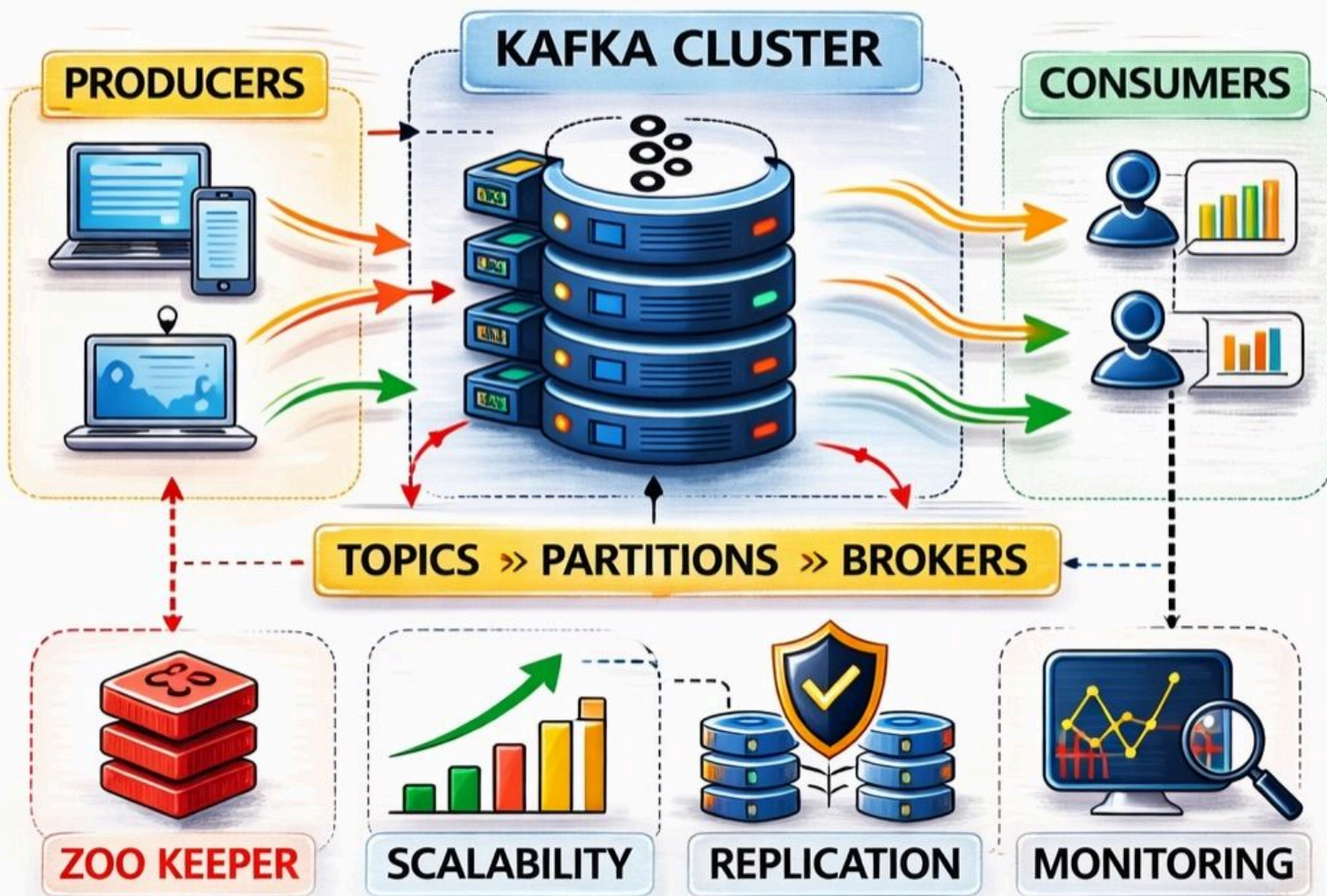


Kafka

Complete Interview Series

follow [@jobtechmingle](#) →

Learn • Understand • **Master** • **Crack Interviews**



From **Basics** → **Architecture** →
Performance → **Interview Q&A**

→ Swipe → Save for Revision  / Share

What is Apache Kafka & Why It Exists

follow [@jobtechmingle](#) →

What is Apache Kafka?

Apache Kafka is a **distributed event streaming platform** used to publish, store, and process data streams in real time.

It allows systems to send and receive events reliably at high scale, without being tightly coupled.



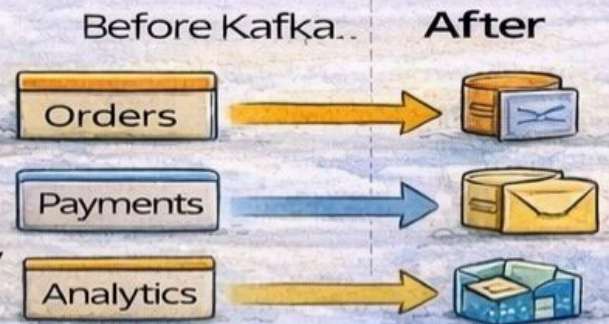
Why Kafka Was Created

Traditional systems struggle when:

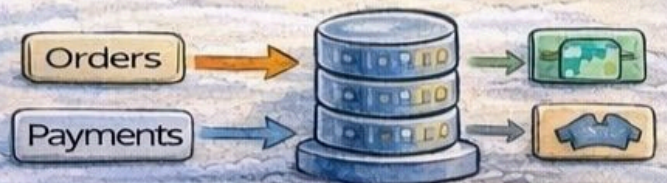
- ✓ Data needs to be processed **in real time**
- ✓ Multiple services depend on the **same data**
- ✓ High volume events must be handled **reliably**

Kafka was built to solve:

- ✓ Real-time data streaming
- ✓ High-throughput messaging
- ✓ System decoupling
- ✓ Scalable event processing



Kafka in Modern Architecture



Problems Kafka Solves

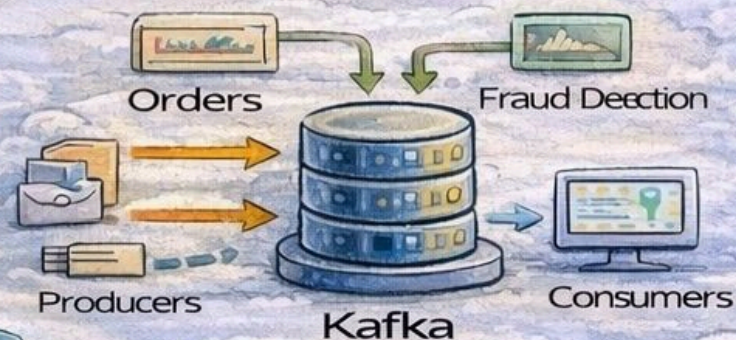
Removes **tight coupling** between producers and consumers

- ✓ Handles **millions** of events per second
- ✓ Enables **replay of past data**
- ✓ Works reliably even when systems fail

Simple Kafka Flow (Conceptual)



(Event driven, asynchronous, scalable)



Kafka is not just a message queue – It is a durable, scalable event log for real-time systems

Kafka Core Architecture

follow @jobtechmingle →

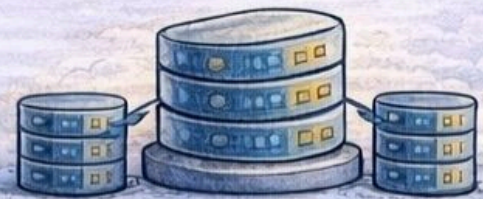
1 Broker

A **Broker** is a Kafka server that:

- ✓ Stores messages
- ✓ Handles read & write requests
- ✓ Manages **partitions**

A Kafka cluster contains **multiple brokers** for scalability and fault tolerance.

Think of a broker as a data storage server in the cluster.

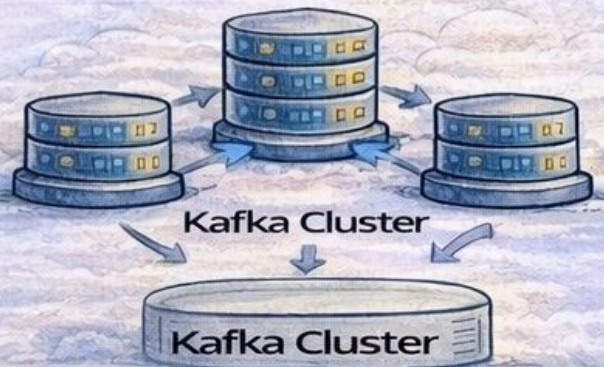


2 Cluster

A **Kafka Cluster** is a group of brokers working together.

- ✓ High availability
- ✓ Load distribution
- ✓ Fault tolerance
- ✓ Horizontal scaling

If one broker fails → others continue serving data.



3 Topic

A **Topic** is a logical category of messages

Examples:

- orders
- payments
- user-clicks



Producers write to topics.

Consumers read from topics.

Important: Topics do not store data in one block — they are split into partitions.

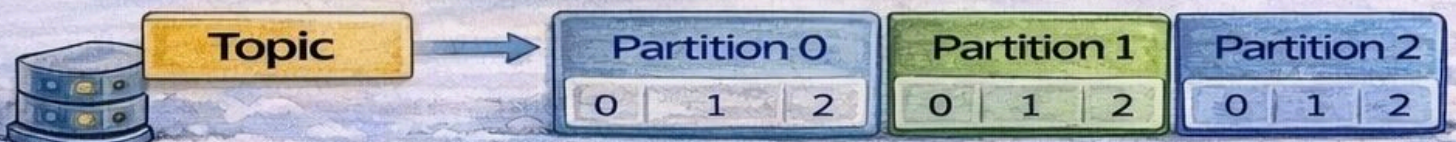
4 Partition

A **Partition** is a unit of parallelism in Kafka.

- ✓ A topic is divided into **multiple** partitions
- ✓ Each partition stores messages in order
- ✓ Ordering is guaranteed only **within a partition**

5 Architecture Flow (Conceptual)

Topic → Partition 0 → Partition 1 → Partition 2



Each partition lives inside brokers, Each message has an **offset**.

Kafka Replication & Fault Tolerance

follow @jobtechmingle →

1 Why Replication is Needed

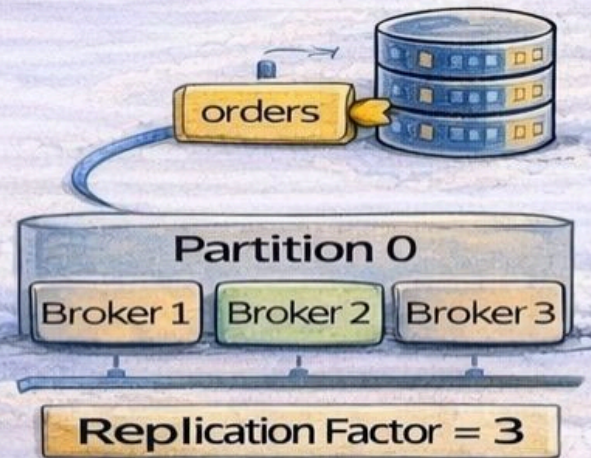
Kafka is built for:

- ✓ High availability
- ✓ Fault tolerance
- ✓ Zero data loss (when configured properly)

If a broker crashes →

Data must still be available.

That's where **replication** comes in.



2 What is Replication?

Each partition can have **multiple** copies across brokers.

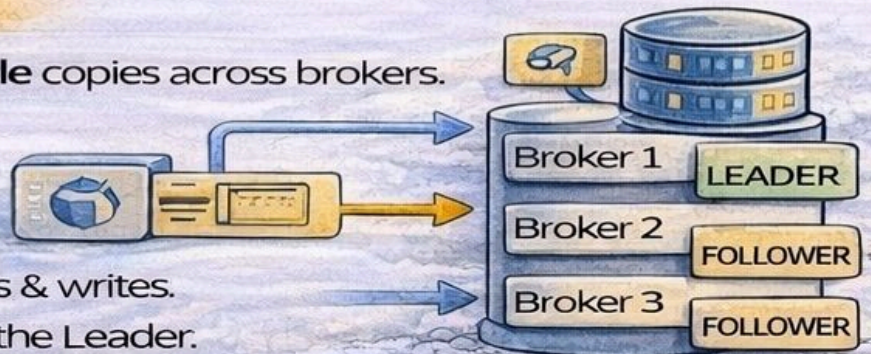
Example:

Topic → Partition 0

Replication Factor = 3

Only the **Leader** handles reads & writes.

Followers **replicate** data from the Leader.



3 Leader & Follower Concept

Every partition has:

- ✓ 1 Leader
- ✓ Multiple Followers

Producer → writes to **Leader**

Consumer → reads from **Leader**
(by default)

Followers continuously sync data.



4 In-Sync Replica (ISR)

ISR = List of replicas that are fully synced with the Leader.

If a follower lags too much →

- ✓ It is removed from ISR.
- ✓ System continues without downtime.

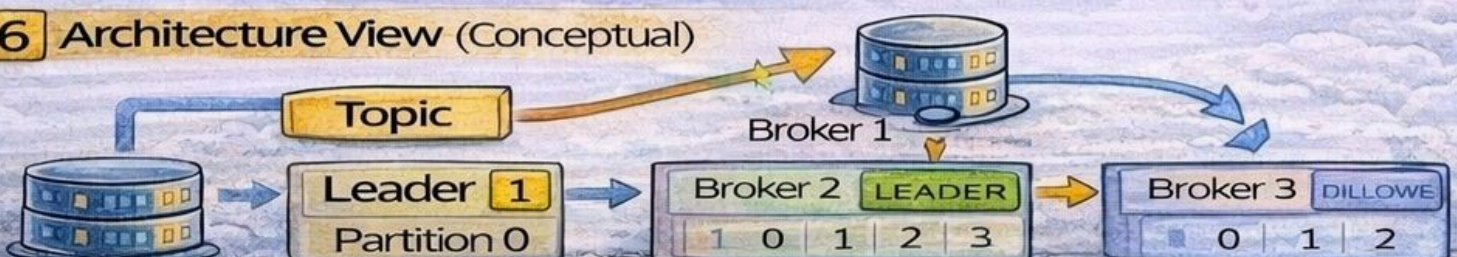
This makes Kafka highly **resilient**.

5 What Happens If Leader Fails?

If Leader crashes:

- ✓ Kafka **elects** a new Leader
- ✓ New **Leader** must be from **ISR**
- ✓ System continues without downtime.

6 Architecture View (Conceptual)



Broker

If Broker 1 fails; Broker 2 becomes Leader.

Kafka Producers – Message Flow & Guarantees

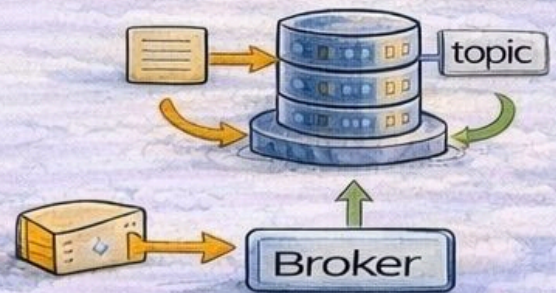
follow @jobtechmingle →

1 What is a Kafka Producer?

A **Producer** is an application that:

- ✓ Publishes data (messages) to Kafka
- ✓ Sends records to a specific **topic**.
- ✓ Decides how and where data is written

Producers are the entry point of data into Kafka.



Kafka Broker (Leader)

2 Producer Message Flow

Typical producer flow;

- ✓ Selects Topic
- ✓ Chooses Partition →
- ✓ Sends Message

Message is written **ONLY** to the Leader partition.

4 Message Guarantees (Delivery Semantics)

Kafka supports three delivery modes:

- ✓ **At-most-once**
No retry, possible loss.
- ✓ **At-least-once**
Retry enabled; duplicates possible

Most real systems use at-least-once.

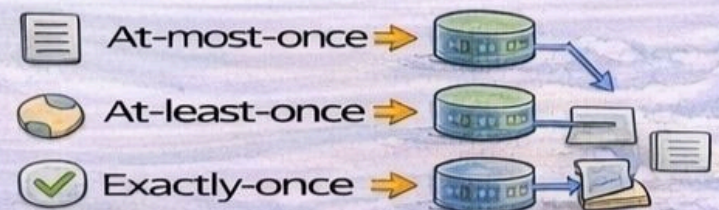
3 How Kafka Chooses a Partition

Kafka uses partitioning logic::

- ✓ Key provided
Same key – Same partition
- ✓ No key
Round robin / Sticky partitioning

Why this matters?

- ✓ Ordering guarantee per partition
- ✓ Parallel processing across partitions



Most real systems use at-least-once.

5 Producer Acknowledgments (acks)

Producer durability control:

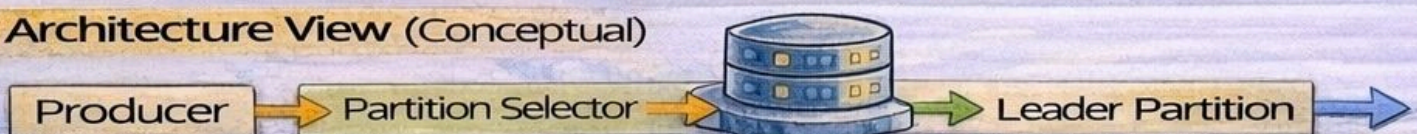
- ✓ acks=0 → Fast, unsafe
- ✓ acks=1 → Leader confirms write
- ✓ acks=all → All ISR replicas confirm

Higher acks = More safety, slightly more latency.

6 Producer Retries & Idempotence

- ✓ Retries help recover from temporary failures
- ✓ Idempotent producer prevents duplicate writes.

7 Architecture View (Conceptual)



Guarantees depend on acks + retries + idempotence

Kafka producers control

Kafka Consumers & Consumer Groups

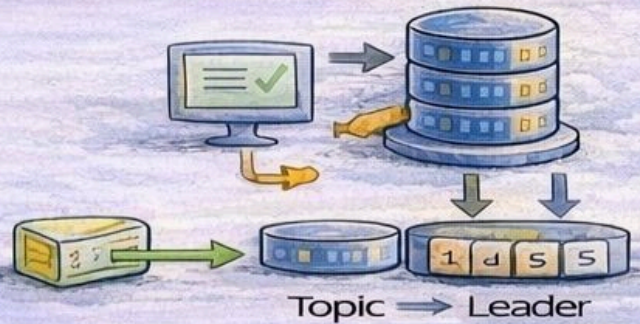
follow @jobtechmingle →

1 What is a Kafka Consumer?

A **Consumer** is an application that:

- ✓ Reads messages from **Kafka topics**
- ✓ Subscribes to one or more topics
- ✓ Processes data in real time.

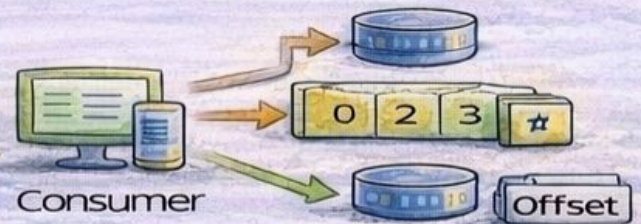
Consumers pull data from leader partitions.



2 Consumer Message Flow

Basic flow:

- ✓ Consumer → Subscribes to Topic
- ✓ Reads from Partition
- ✓ Processes Message.



Consumers control how far they have read using **offsets**.

3 What is a Consumer Group?

A **Consumer Group** is a set of consumers working together.

Why groups exist?

- ✓ Parallel processing
- ✓ Scalability
- ✓ Load balancing



Each partition is consumed by only one consumer per group.

4 Partition Assignment Rule

If Topic has 3 partitions
and Consumer Group has 2 consumers:

Distribution:

- ✓ Consumer A → Partition 0 & 1
- ✓ Consumer B → Partition 2

If consumers > partitions
Extra consumers stay idle.



5 Offset Management

Offsets track the last processed message.

Kafka stores offsets in:

→ `consumer_offsets` topic.



Auto commit → Manual commit

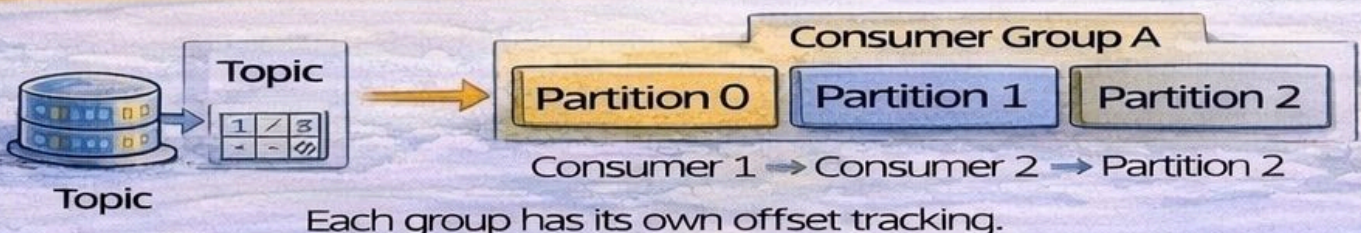
6 Rebalancing

- ✓ A consumer joins
- ✓ A consumer leaves
- ✓ A consumer crashes

Kafka redistributes partitions across group members.



7 Architecture View (Conceptual)



Consumer Groups enable Kafka to scale horizontally while preserving partition-level ordering.

Kafka Consumer Groups & Offsets

follow @jobtechmingle →

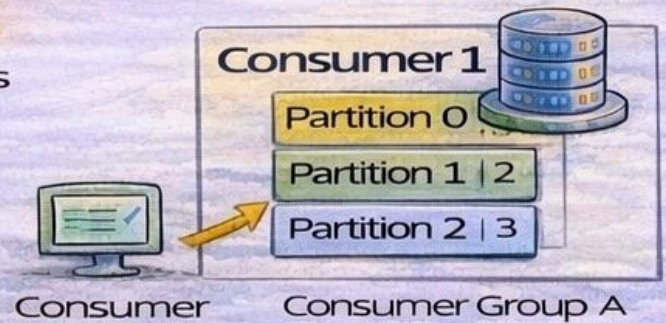
1 What is a Consumer Group?

A Consumer group is a set of consumers working together

- ✓ Each partition is consumed by only **ONE** consumer in a group

- ✓ Enables parallel processing & scalability

One group = one logical application



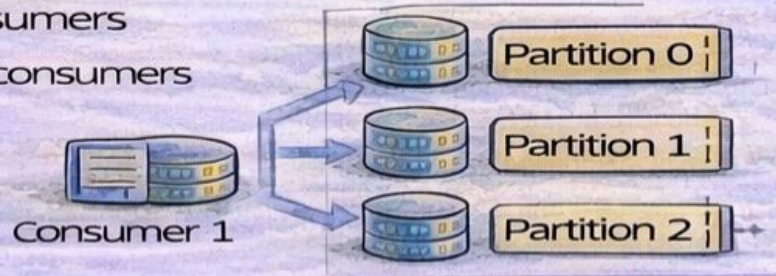
2 How Partition Assignment Works

Kafka assigns partitions among consumers

- ✓ If partitions > consumers → some consumers handle multiple partitions

- ✓ If consumers > partitions → some consumers stay idle

One group = one logical application

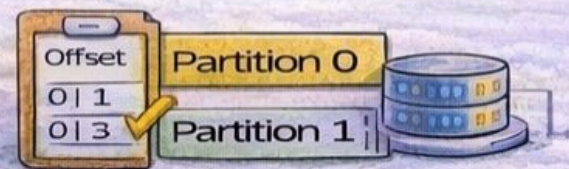


3 What are Offsets?

Offset = position of a message inside a partition

- ✓ If partitions > consumers

- ✓ Kafka remembers last committed offset.



4 Offset Commit Types

- ✓ Auto Commit

- ✓ Kafka commits automatically at intervals

- ⚠ Risk; message loss or duplication

- ✓ Manual Commit

- ✓ Commit only after processing

- ✓ Safer for critical pipelines

5 Rebalancing Explained

Rebalance happens when:

- ✓ New consumer joins

- ✓ Consumer crashes

- ✓ Topic partitions change.



During rebalance:

- ✓ Partitions are reassigned

- ✓ Consumers briefly pause

6 At-Least vs Exactly-Once (Concept)

- ✓ At-least-once

- ✓ possible duplicates

- ✓ Exactly-once

transactional processing (advanced)



? Real Interview Scenario

- ✓ Offsets not committed properly

- ✓ Consumer group ID changed

- ✓ Auto commit delay



Kafka scalability comes from consumer groups, reliability comes from correct offset management.

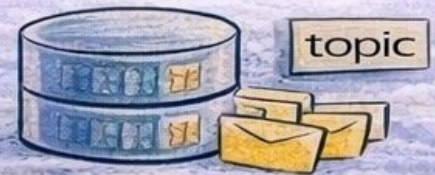
Topics, Partitions & Throughput

follow @jobtechmingle →

1 What is a Topic?

A **Topic** is a logical stream of records in Kafka.

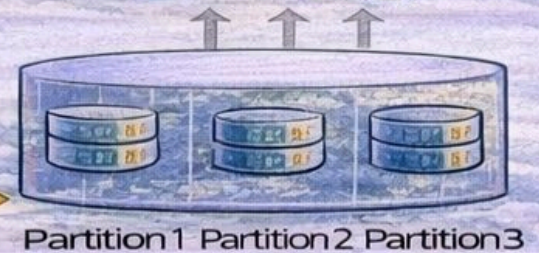
- ✓ Similar to a **table name**, but data is immutable
- ✓ Topics do not process data = they only store it.



2 What is a Partition?

A **Partition** is a physical split of a topic.

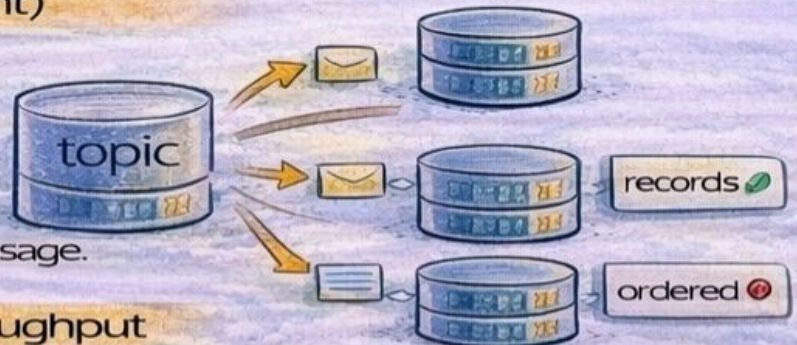
- ✓ Each partition is an ordered, immutable log
- ✓ Messages are written **sequentially** with increasing offsets.



3 Ordering Guarantee (Important)

Multiple partitions allow:

- ✓ Parallel writes by producers
- ✓ Parallel reads by consumers
- ✓ More partitions **≠** higher throughput, not faster single message.

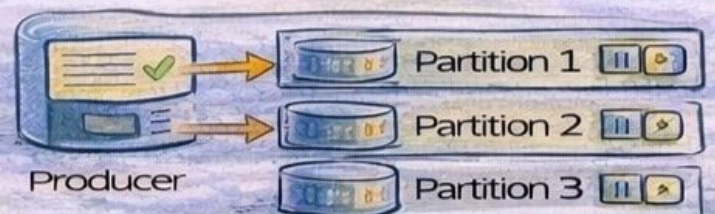


4 How Partitions Improve Throughput

Multiple partitions allow:

- ✓ Parallel writes by producers
- ✓ Parallel reads by consumers
- ✓ More partitions = higher throughput, not faster single message.

Partitions only within a partition



5 Key-Based Partitioning

- ✓ Messages with the **same key** always go to a **same partition**.
- ✓ Ensures order for events related to the same entity.

6 Key-Based Partitioning

- ✓ Messages with the **same key**
- ✓ Key → # → 1 → 3
- ✓ No Key → # → 3

Partitions increase **parallelism**.



Partitions are the core scaling unit in Kafka.

Throughput increases by adding partitions —
— not by speeding up brokers.

Kafka Delivery Semantics & Exactly-Once Processing

follow @jobtechmingle →

1 What Are Delivery Semantics in Kafka?

Kafka defines how many times a message is delivered from producer to consumer.

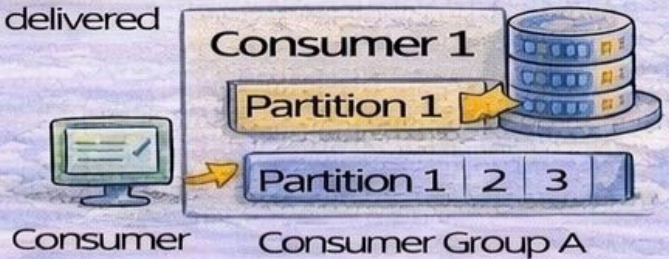
- ✓ Kafka supports three delivery guarantees, depending on configuration.

1 At-Most-Once

- ✓ Message is sent without retry
- ✓ If failure occurs → message may be lost
- ✓ Fastest, but least reliable

Used when:

- ✓ Logs, metrics, non-critical data



2 At-Least-Once

- ✓ Message is retried on failure
- ✓ Message is never lost
- ✓ Duplicates possible



3 Exactly-Once (EOS)

- ✓ Message is processed only once
- ✓ No loss, no duplication
- ✓ Requires Kafka Transactions

Used when:

- ✓ Data pipelines, analytics ingestion



How Exactly-Once Works (Conceptual Flow)

- 1 : Producer starts a transaction
- 2 : Writes messages to Kafka
- 3 : Commits transaction



(Aborted transactions are invisible)

4 What Are Kafka Transactions?

Kafka transactions ensure:

- ✓ Atomic writes across multiple partitions
- ✓ Safe producer retries
- ✓ Exactly-once delivery between producer → Kafka → Consumer

Key Components Involved

- ✓ Idempotent Producer
- ✓ Transactional ID
- ✓ Transaction Coordinator
- ✓ Committed Offset Management



⚠ Important Limitations

- ✓ Slight performance overhead
- ✓ More complex configuration
- ✓ Requires careful producer & consumer

? Interview Insight

- Exactly-once in Kafka means exactly-once processing, not exactly-once delivery over the network.



Kafka gives you flexibility:
Speed vs Reliability vs Accuracy —
you choose via configuration.

Kafka Performance Tuning & Best Practices

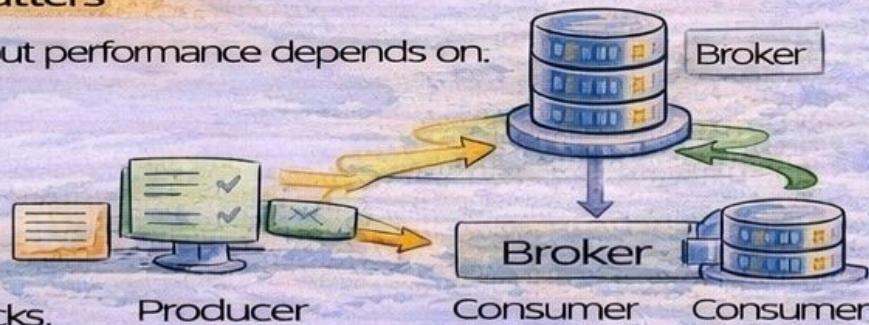
follow @jobtechmingle →

1 Why Performance Tuning Matters

Kafka is built for high throughput; but performance depends on:

- ✓ Producer configuration
- ✓ Broker setup
- ✓ Consumer tuning
- ✓ Hardware & partition strategy

Improper configuration → bottlenecks.



2 Producer Performance Tuning

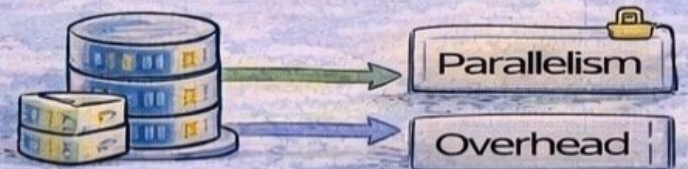
Key configs:

- ✓ batch.size → Larger batch = higher
- ✓ linger.ms
- ✓ compression type → gzip / snappy / lz4
- ✓ acks → Trade-off between speed & durability

Optimization Tip. Use *batching* + *compression* for high-volume systems.

3 Partition Strategy

- ✓ More partitions – More parallelism
- ✓ But too many partitions – overhead

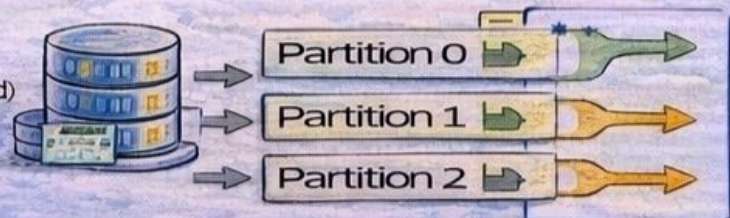


Parallelism | Overhead

4 Broker-Side Tuning

- ✓ Increase replication factor (usually 3)
- ✓ Proper disk configuration (SSD recommended)
- ✓ Tune log retention settings
- ✓ Monitor disk I/O & network bandwidth

Kafka performance is disk + network bound.



5 Consumer Performance Tuning

- ✓ fetch.min.bytes
- ✓ max.poll.records
- ✓ session.timeout.ms

Scale consumers horizontally using consumer groups.



6 Monitoring & Metrics

- ✓ Consumer lag
- ✓ Under-replicated partitions
- ✓ Request latency

Prometheus + Grafana / Grafana Control Center



7 Common Performance Mistakes

- ✗ Too few partitions
- ✗ Excessive small messages
- ✗ No compression
- ✗ Auto commit in critical systems
- ✗ Ignoring consumer lag



Interview Insight

Kafka performance is usually limited by:

Disk I/O, Network throughput, or Partition design — not CPU.

Kafka – Real Interview Scenarios & Debugging Guide

follow @jobtechmingle →

1 Scenario: Consumer Lag Keeps Growing

What interviewer wants to test:
Understanding throughput vs processing capacity.

Root Causes:

- ✓ Fewer partitions than consumers
- ✓ Heavy message processing logic.
- ✓ Small fetch / poll configuration
- ✓ Network or disk bottleneck

Correct Fixes:

- ✓ Increase topic partitions
- ✓ Scale consumer group horizontally
- ✓ Optimize message processing



2 Scenario: Messages Re-Consumed After Restart

What it indicates:

Offset handling issue (very-common-interview trap)

Root Causes:

- ✓ Offsets not committed after processing
- ✓ Auto-commit misconfigured
- ✓ New consumer group ID used

Correct Fixes:

- ✓ Commit offsets “after” successful processing
- ✓ Use manual commit for critical pipelines
- ✓ Maintain stable consumer group IDs



3 Scenario: Missing / Lost Messages

What interviewer checks:
Producer reliability knowledge.

Root Causes:

- ✓ acks = 0 or acks = 1
- ✓ Low replication factor
- ✓ Leader crash before replication

Correct Fixes:

- ✓ Use acks = all
- ✓ Replication factor ≥ 3
- ✓ Monitor ISR shrink events
- ✓ Validate retention.ms & retention.bytes



4 Scenario: One Consumer Overloaded

Whats happening:

Uneven partition distribution

Root Causes:

- ✓ Use high-cardinality keys
- ✓ Balance hashing strategy
- ✓ Re-partition if data is skewed



6 Practical Kafka Debugging Checklist.

- ✓ Check broker health & ISR count
- ✓ Monitor consumer lag metrics
- ✓ Verify partition, to consumer mapping
- ✓ Inspect producer acks & retries

6 Practical Kafka Debugging Checklist

- ✓ Check broker health & ISR count
- ✓ Monitor consumer lag metrics
- ✓ Verify partition, to consumer mapping
- ✓ Inspect producer acks & retries

Kafka failures are rarely caused by Kafka itself, not servers.



Kafka failures are rarely “Kafka bugs” — not servers.